Predictive Maintenance for Sustainable Water Infrastructure: A Comparative Machine Learning Study of the Tanzania Water Pump Dataset

Course: Digital Technology (DT2) Author: *(Chenyun Cui)*
Student number: *( )*



AI use disclosure: Claude was used to support grammar checking and editing, double-column formatting, code review.ChatGPT was used to generate Pipeline diagram by hand-designed prompt

Abstract Tanzania operates over 59,000 rural water extraction points, many of which are non-functional or in need of repair. This study applies machine learning to the Taarifa/Ministry of Water dataset to predict pump status across three classes: *Functional*, *Non-Functional*, and *Needs Repair*. Missing a failing pump costs more than a wasted inspection, so I built a social cost function ($C_{\text{social}} = 5 \times FN_{\text{Class 2}} + 1 \times FP_{\text{Class 2}}$) that weights missed repairs five times higher than false alarms.

I compared XGBoost against a Multi-Layer Perceptron (MLP). XGBoost, with a Class 2 weight of 5, achieved the lowest social cost of 2,752 (Class 2 recall = 0.6385; Class 1 precision = 0.8750; accuracy = 0.7507). The tuned MLP came close, with a social cost of 2,879 (Class 2 recall = 0.5944; Class 1 precision = 0.8351; accuracy = 0.7501). Both models satisfied most technical requirements, but XGBoost performed better on the cost-sensitive evaluation metrics. Recent benchmarks likewise show that tree-based models continue to outperform neural networks on medium-sized tabular datasets (Grinsztajn et al., 2022).

Keywords: Social sustainability; predictive maintenance; XGBoost; multi-layer perceptron; class imbalance; water infrastructure.

I. PROBLEM DEFINITION & BACKGROUND

A. Application Scenario

Tanzania's rural water network spans tens of thousands of extraction points, including hand pumps, motorised wells, gravity-fed systems, and communal standpipes, under varied geographic and climatic conditions. The Ministry of Water and the Taarifa open-data platform collect metadata for each installation: construction year, water source type, extraction mechanism, payment structure, GPS coordinates, and managing authority.

The core task is a multi-class classification problem: given the metadata of a water pump, predict its operational status as one of three classes:

Class	Label	Prevalence
0	Functional	~54%
1	Non-Functional	~39%
2	Needs Repair	~7%

B. Construction of Social-cost function

The project relates to UN Sustainable Development Goal 6 (Clean Water and Sanitation). If the model misses a pump that needs repair (a false negative on Class 2), residents lose access to clean water. Sending a crew to a pump that turns out fine only wastes budget. "Needs Repair" is only about 7% of the data, so models tend to ignore it unless forced.

To reflect this imbalance, I set up a cost function:

$$C_{\text{social}} = 5 \times F_{N,\text{Class 2}} + 1 \times F_{P,\text{Class 2}}$$

Industry data (e.g., Buildings.com) shows reactive emergency repair typically costs 3–5× more than routine maintenance. I picked the upper end of that range but did not go higher, because pushing the penalty too far makes the model flag everything as "needs repair" and the false positive count increases sharply.

II. REQUIREMENTS & CONSTRAINTS

A. Problem components

1. Ministry of Water: Maximise recall for "Needs Repair" to prevent humanitarian crises.
2. Budget Office: Maintain high precision for "Non-Functional" to justify dispatch costs.
3. Field Engineers: Require explainable predictions and confidence scores.

B. Technical Requirements (TR)

I defined five technical requirements that any candidate model must satisfy:

ID	Requirement	Threshold	Rationale
TR-01	Class 2 Recall	> 0.50	Detect majority of pumps needing repair
TR-02	Class 1 Precision	> 0.80	Avoid wasting budget on healthy pumps
TR-03	Overfitting Gap	< 0.10	Model must generalise to unseen data
TR-04	Probabilistic Confidence	output $P(y \mid x)$	Enable confidence scores
TR-05	Class 2 Lift	> 3.0×	Substantially better than random

TR-01 requires the model to catch at least half the pumps that need repair. DrivenData community analyses show that basic models only manage about 34% recall on Class 2, so 50% is already a meaningful improvement over baseline.

TR-02 is about not wasting money. When the model labels a pump "Non-Functional", it should be right at least 80% of the time, because each field dispatch costs the government real budget.

TR-03 guards against overfitting. I set the threshold at 0.10, meaning the gap between training and validation accuracy must

stay under 10 percentage points. For tabular data with 60K samples this is a reasonable bar.

TR-04 A prediction like "72% chance this pump needs repair" is much more useful for field teams than a flat yes/no, because they can prioritize actions based on urgency.

TR-05 measures lift: how much better is the model at finding Class 2 pumps compared to random guessing? A lift of 3× means the model concentrates its Class 2 predictions three times more effectively than chance would.(typically,thresholds above 4× tend to increase false positives substantially while recovering few additional Class 2 samples at this class distribution.)

C. Verification Plan

Step 1: Stratified data split. A single stratified 80/20 holdout split will be created via `train_test_split(..., stratify=y, random_state=42)`.Stratification preserves the 7.27% Class 2 prior across both folds, so both models are evaluated on the same physical pumps. K-fold will not be used: at this scale, five retraining runs per model add negligible improvement over a single well-sized holdout. Feature representations differ slightly: XGBoost directly consumes the pre-engineered feature matrix, MLP applies a `StandardScaler` and `OneHotEncoder` fitted on training data only. The validation samples are otherwise identical, so performance differences reflect model architecture and hyperparameters.

Step 2: Train and predict. Each model will generate predicted labels and class probabilities via `predict_proba` on the shared validation set. XGBoost will be evaluated at four Class 2 weight settings (1×, 5×, 10×, 20×),the MLP will be tested across ten (α, learning rate) combinations. Each configuration is trained independently from scratch.

Step 3: Confusion matrix extraction. For each configuration, a 3 × 3 confusion matrix will be created by `sklearn.metrics.confusion_matrix`. Two quantities are extracted: $FN_{\text{Class 2}}$, counting pumps requiring repair that are assigned a Functional or Non-Functional label, and $FP_{\text{Class 2}}$, counting pumps labelled "Needs Repair" that belong to another class.

Step 4: Metric computation. `sklearn`'s built-in metrics, such as accuracy and F1, do not represent the 5:1 asymmetric social cost, so evaluation uses a custom audit script. For each configuration it computes Social Cost = $5\times FN_{\text{Class 2}} + 1\times FP_{\text{Class 2}}$ as the primary objective, read directly from confusion matrix cells. Selection is based on five technical requirements: Class 2 recall above 0.50 (TR-01), Class 1 precision above 0.80 (TR-02), overfitting gap below 0.10 (TR-03), softmax probability outputs present (TR-04), and Class 2 lift above 3.0× (TR-05). The configuration with the lowest Social Cost that satisfies all five will be selected, and results will be written to `audit_results.csv`.

III. MODEL SUITABILITY EVALUATION & SELECTION

A. Why I Chose XGBoost and MLP

I needed one classical ML model and one neural network. Starting from the requirements:, I went through the options:

1. Logistic Regression: A simple baseline, but I did not select it as one of the final models because the main task is not just to separate the three classes linearly. The "Needs Repair" class depends on combinations of factors such as pump age, water source type, region, and payment behaviour. For example, an old pump in a hotspot region using surface water is more informative than any one of those variables alone. Logistic Regression would require me to manually define many interaction terms to capture this kind of risk pattern.

2. Decision Tree: It is too unstable for this case study. The dataset contains many categorical and geographic features, so one tree could overfit to specific regions, funders, or installers in the training data.

3. Random Forest: Random Forest was considered as a strong classical baseline because it works well on tabular data and reduces the instability of a single Decision Tree. However, I selected XGBoost instead because my main design problem was not just prediction accuracy, but controlling the trade-off between missed "Needs Repair" pumps and unnecessary inspections. `sample_weight` support, allowing me to assign 5× higher importance to Class 2 samples so the model aligns with the social cost objective.

4. XGBoost: Ultimately, I chose it as the classical model. It requires minimal preprocessing to handle mixed features, comes with built-in `sample_weight` support for cost-sensitive learning, and outputs feature importance for easy interpretation. Training is also fast, taking only about 120 seconds. Community results on the DrivenData dataset also show that it performs well.

5. MLP: It required more preprocessing than XGBoost, including standardisation, one-hot encoding, and Class 2 oversampling. This made it useful as a contrast: XGBoost addressed the rare "Needs Repair" class through `sample_weight`, while the MLP addressed it by changing the training distribution through physical oversampling.

IV. DATASET & EXPLORATORY DATA ANALYSIS

A. Data Source

The dataset comes from the DrivenData "Pump it Up" competition, sourced from the Tanzanian Ministry of Water via the Taarifa waterpoints dashboard. It contains:

1. Training set: 59,400 water extraction points with 40 features and a target label.

2. Test set: 14,850 observations (labels withheld by competition).

3. Target variable: `status_group` encoded as {0: Functional, 1: Non-Functional, 2: Needs Repair}.

B. Class Distribution & Imbalance

The target variable is imbalanced:

Class	Count	Percentage
Functional (0)	~32,259	54.31%
Non-Functional (1)	~22,824	38.42%
Needs Repair (2)	~4,317	7.27%

Class 2 ("Needs Repair") is only 7.27% of the data, the class I most need to detect and the hardest for any model to learn.

C. Data Cleaning Pipeline (*data_cleaning.py*)

I cleaned the data in six targeted steps:

1. I merged the training values with labels on id and concatenated the train and test sets (using an _is_train flag) so that all categorical encodings remain consistent across both sets.

2. I dropped columns that carry no predictive information: num_private (no variance), scheme_name (high missingness), quantity_group (redundant with quantity), and wpt_name (excessive cardinality, acting like an identifier).

3. Longitude values of 0 and GPS heights of 0 are physically impossible, so I replaced them with NaN. No imputation was performed,the missingness itself may carry signal.

4. High-cardinality categorical features (funder and installer) were grouped into their Top-20 most frequent categories plus an "Other" bucket to keep the feature space manageable for subsequent encoding.

5. I mapped the target variable to integers: functional = 0, non functional = 1, functional needs repair = 2.

6. I preserved zero values in amount_tsh and population as informative risk signals and filled remaining missing categorical values with "Unknown". For construction_year, 0 does not mean the pump was built in year 0; it means the year is missing. I kept this as a separate missing-year flag because missing construction records may indicate older or poorly documented infrastructure, which can be linked to failure risk. At the same time, pump_age cannot be calculated using 0, because that would produce an unrealistic age. Therefore, for the pump_age feature only, missing construction years were replaced with the median year, 2000.

D. Strategic EDA Findings (*strategic_eda.py*)

I ran a risk-oriented EDA to find features that separate Class 2 from the rest. Four types of signal emerged:

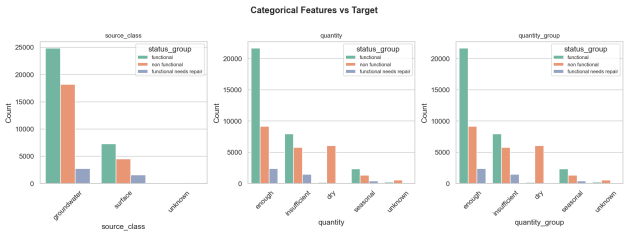


Figure 1: Sure-kill signals identified in the strategic EDA.

Sure-kill signals (>90% failure risk): When quantity == 'dry' or payment_type == 'never pay', pumps almost always fail. I turned these into binary flags (is_dry_well, is_never_pay).

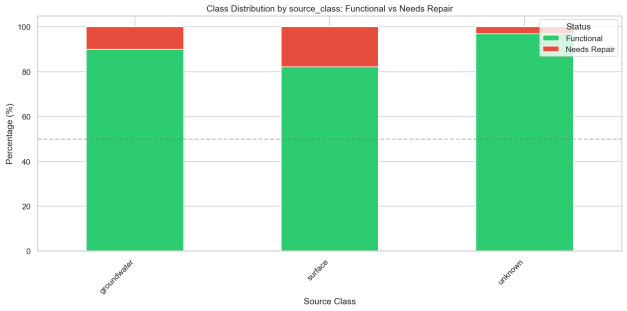


Figure 2: Hidden risk signals identified in the strategic EDA.

Hidden risk signals: source_class == 'surface' turned out to be the best single feature for separating Functional from Needs Repair. I created is_surface_water from it.

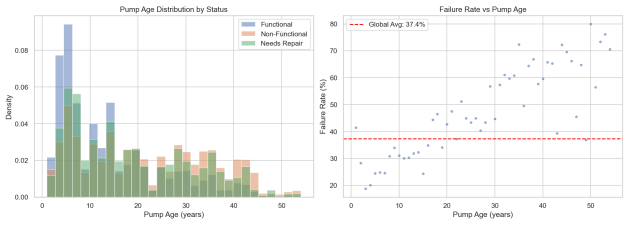


Figure 3: Temporal signals identified in the strategic EDA.

Temporal signals: Pumps older than 20 years fail much more often. Also, construction_year == 0 (missing) is itself a risk indicator. I created pump_age and is_old_pump features.

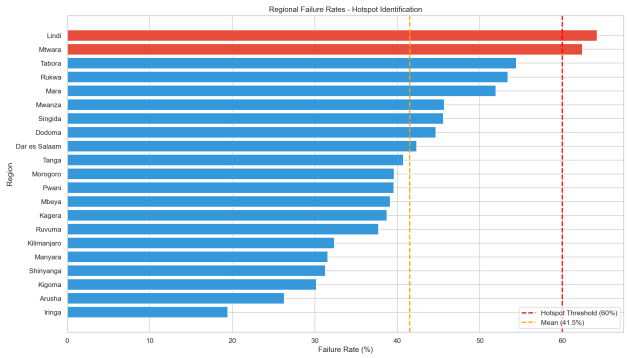


Figure 4: Geospatial hotspots identified in the strategic EDA.

Geospatial hotspots: Regions Lindi and Mtwara have failure rates above 60%. I flagged these with is_hotspot_region.

E. Numerical Feature Analysis (strategic_eda.py)

I also analysed the 10 numerical features to understand their distributions and decide on preprocessing.

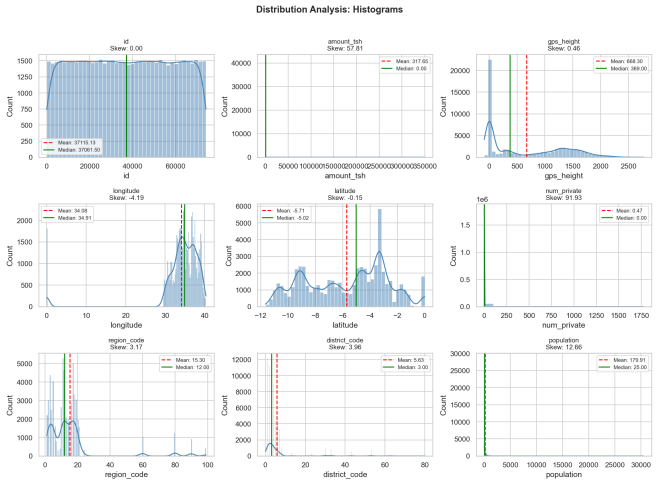


Figure 5: Histograms of the numerical features.

Key findings from the histograms:

1. amount_tsh has extreme right skew (most values are 0), meaning payment data is mostly missing.

2. longitude has a spike at 0 (~1,800 samples), confirming the need for physics-based imputation.

3. population has extreme outliers (up to 30,000), which means the MLP will need scaling to handle these properly.

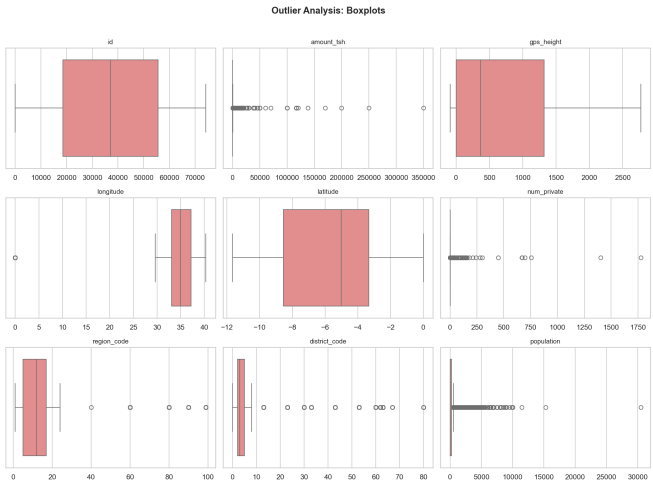


Figure 6: Boxplots showing outliers across the numerical features.

The boxplots confirmed outliers across most numerical columns. For XGBoost this is not a problem, since tree splits are invariant to monotonic transformations and outliers. The MLP sees raw numbers, so I applied StandardScaler to prevent these extreme values from dominating gradient updates.

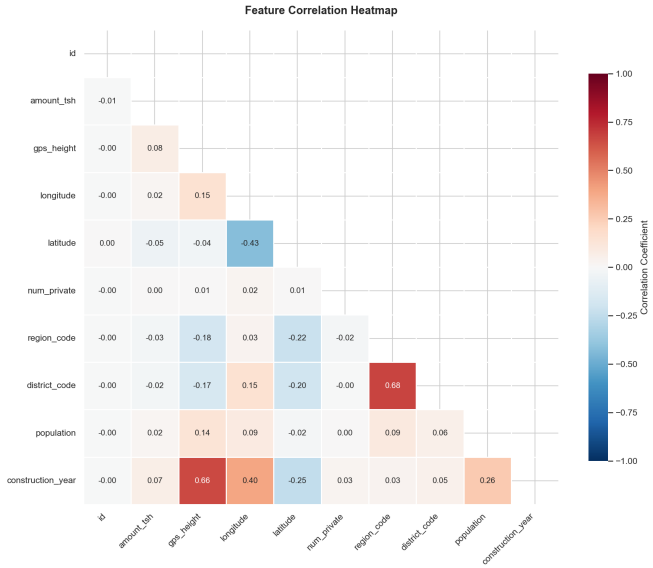


Figure 7: Correlation heatmap for the numerical features.

The correlation heatmap was mostly flat; nearly all pairwise correlations sit close to zero. The one exception is region_code vs. district_code ($r = 0.68$), which is expected since districts nest inside regions. The low overall correlation told me that PCA would not help much here. The features carry largely independent information, so I kept them all.

F. Data Partitioning Strategy

I split the labelled training data 80/20 using train_test_split with stratify=y and random_state=42. Stratification matters here because

Class 2 is only 7.27% of the data, so a random split could easily under-represent it in one fold. Using a fixed seed also means both models see the exact same validation samples, which makes the comparison fair and repeatable.

The two models handle the split differently, and this matters:

1. XGBoost preprocesses before splitting. It only drops text columns and fills NaN with 0, operations that use no statistics from the data (no mean, no variance), so there is no leakage risk.

2. MLP splits first, then preprocesses. The StandardScaler and OneHotEncoder are fitted on the training fold only and then applied to the validation fold. If I had fitted the scaler on all data, validation statistics would leak into training.

V. PIPELINE OVERVIEW

The pipeline has three shared stages, then forks into separate paths for XGBoost and MLP:

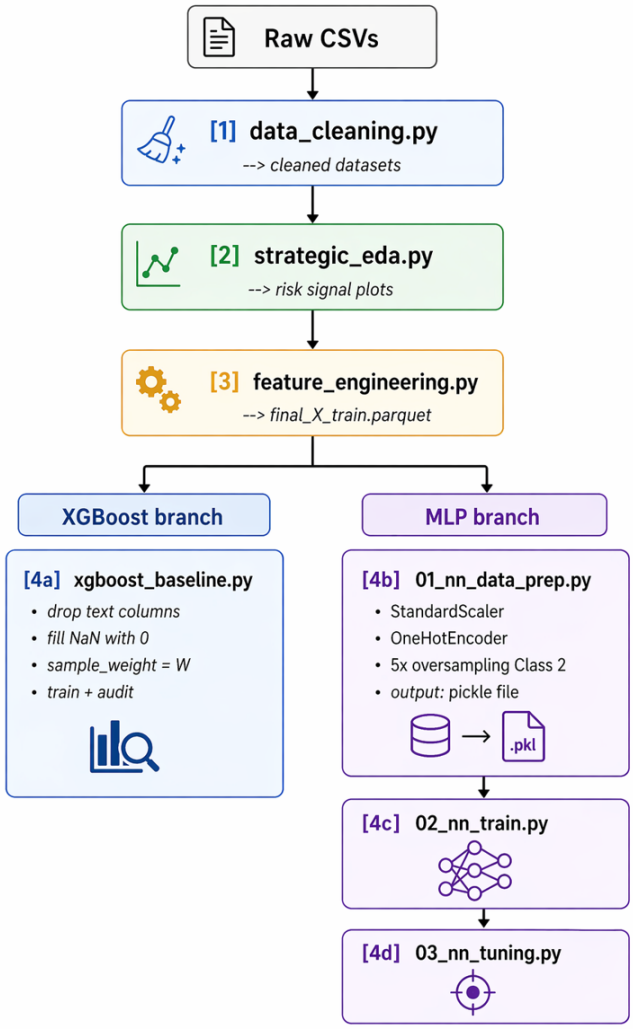


Figure 8: Pipeline overview

A. *Shared stages (1–3)*

Stage 1: Cleaning (data_cleaning.py). Training values and labels are merged on id; train and test sets are concatenated with an _is_train flag so categorical encodings remain consistent across splits. Four columns are dropped: num_private (zero variance), scheme_name (high missingness), quantity_group (duplicate of quantity), and wpt_name (cardinality too high to encode meaningfully). Physically impossible values — gps_height = 0, longitude = 0 — are set to NaN without imputation; the missingness pattern may itself carry signal. High-cardinality categoricals funder and installer are reduced to Top-20 plus "Other".

Stage 2: EDA (strategic_eda.py). Four risk signals identified from the EDA: sure-kill conditions (quantity == 'dry', payment_type == 'never pay'), a hidden discriminator (source_class == 'surface'), temporal decay encoded as pump_age, and geographic concentration in Lindi and Mtwara.

Stage 3: Feature engineering (feature_engineering.py). EDA findings are translated into model-ready features:

Feature	Why I created it
is_dry_well, is_never_pay	Sure-kill signals with >90% failure risk
is_surface_water	Best discriminator between Functional and Needs Repair
pump_age, is_old_pump	Pumps older than 20 years fail much more often
is_hotspot_region	Lindi and Mtwara have >60% failure rates
One-hot encoding	Converted all remaining categoricals to binary columns

The output is a feature matrix with 2,400+ columns stored as Parquet files. This is the last shared artefact; the pipeline forks here.

B. *XGBoost branch (Stage 4a)*

XGBoost handles mixed features natively, so preprocessing is minimal. I dropped leftover text columns and filled remaining NaN with 0. Neither operation uses any data statistics (no mean, no variance), so there is no leakage risk even though preprocessing happens before the train/validation split.

For class imbalance I used sample_weight: each Class 2 sample receives weight W in the loss function, so errors on those samples cost the optimiser more. This keeps the data distribution untouched.

C. *MLP branch (Stages 4b–4c)*

Neural networks are sensitive to feature scales and cannot handle raw categoricals, so the MLP path requires more preprocessing (01_nn_data_prep.py):

1. First, I split into train/validation, then fit StandardScaler and OneHotEncoder on the training fold only to prevent leakage.

```
    print("\n[STEP 4] Stratified Train/Val Split")
    print("-" * 55)

    X_train_df, X_val_df, y_train, y_val = train_test_split(
        X, y,
        test_size=VAL_SIZE,
        stratify=y,
        random_state=RANDOM_STATE
    )
```

Figure 9: First perform train/validation split.

```
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_cols),
        ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=True), categorical_cols)
    ],
    remainder='drop',
    sparse_threshold=0.3
)

print("   ✓ StandardScaler for numerical features")
print("   ✓ OneHotEncoder for categorical features")

print("\n[STEP 6] Fitting Transformer on Training Data")
print("-" * 55)

X_train_processed = preprocessor.fit_transform(X_train_df)
X_val_processed = preprocessor.transform(X_val_df)
```

Figure 10: Define StandardScaler / OneHotEncoder, then use fit_transform only on X_train_df and transform only on X_val_df.

2. Remove any column with more than 50 unique values before encoding, to keep the sparse matrix manageable.

```
HIGH_CARDINALITY_THRESHOLD = 50
```

Figure 11: The threshold is set to 50.

```
def filter_high_cardinality(df, categorical_cols, threshold):
    valid_cats = []
    dropped_cats = []

    for col in categorical_cols:
        n_unique = df[col].nunique()
        if n_unique > threshold:
            dropped_cats.append((col, n_unique))
        else:
            valid_cats.append(col)

    return valid_cats, dropped_cats
```

Figure 12: Columns where n_unique exceeds the threshold are dropped into dropped_cats.

```
cols_to_drop = [col for col, _ in dropped_cats]
X_train_df = X_train_df.drop(columns=cols_to_drop, errors='ignore')
X_val_df = X_val_df.drop(columns=cols_to_drop, errors='ignore')

numerical_cols, categorical_cols = identify_column_types(X_train_df)
categorical_cols = [c for c in categorical_cols if c in valid_cats]

print(f"   Final feature columns:")
print(f"      Numerical: {len(numerical_cols)}")
print(f"      Categorical: {len(categorical_cols)}")

preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_cols),
        ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=True), categorical_cols)
```

Figure 13: Actual call to high-cardinality filtering

```
cols_to_drop = [col for col, _ in dropped_cats]
X_train_df = X_train_df.drop(columns=cols_to_drop, errors='ignore')
X_val_df = X_val_df.drop(columns=cols_to_drop, errors='ignore')

numerical_cols, categorical_cols = identify_column_types(X_train_df)
categorical_cols = [c for c in categorical_cols if c in valid_cats]

print(f"   Final feature columns:")
print(f"      Numerical: {len(numerical_cols)}")
print(f"      Categorical: {len(categorical_cols)}")

preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_cols),
        ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=True), categorical_cols)
```

Figure 14: Drop these columns before OneHotEncoder.

3. Oversample Class 2 by 5× (literal copies). I considered SMOTE but decided against it: generating synthetic rows from one-hot features can produce combinations that correspond to no real pump.

```
CLASS_2_OVERSAMPLE_FACTOR = 5
```

Figure 15: Set oversample factor = 5

```
def physical_oversample_class2(X, y, factor):
    class2_mask = (y == 2)
    X_class2 = X[class2_mask]
    y_class2 = y[class2_mask]

    class2_count_before = len(y_class2)

    X_oversampled_parts = [X]
    y_oversampled_parts = [y]

    for _ in range(factor - 1):
        X_oversampled_parts.append(X_class2)
        y_oversampled_parts.append(y_class2)

    if sp.issparse(X):
        X_oversampled = sp.vstack(X_oversampled_parts)
    else:
        X_oversampled = np.vstack(X_oversampled_parts)

    y_oversampled = np.concatenate(y_oversampled_parts)
```

Figure 16: Copy Class 2 sample

```
print("\n[STEP 7] Physical Oversampling for Class 2 (FR-01)")
print("-" * 55)
print(f"   Oversampling factor: {CLASS_2_OVERSAMPLE_FACTOR}x")
print(f"   Objective: Minimize 5*FN + 1*FP")

X_train_oversampled, y_train_oversampled, before, after = physical_oversample_class2(
    X_train_processed, y_train, CLASS_2_OVERSAMPLE_FACTOR
)
```

Figure 17: Call Class 2 oversampling

```
    print(f"\n   CLASS 2 VERIFICATION:")
    print(f"      Before oversampling: {before:,} samples")
    print(f"      After oversampling:  {after:,} samples")
    print(f"      Multiplier achieved: {after / before:.1f}x")
```

Figure 18: Validate oversampling multiplier

4. Shuffle the oversampled data so the model does not see all copies back-to-back.

```
shuffle_idx = np.random.RandomState(RANDOM_STATE).permutation(len(y_oversampled))
X_oversampled = X_oversampled[shuffle_idx]
y_oversampled = y_oversampled[shuffle_idx]
```

Figure 19: Oversampled data is shuffled to prevent copies from being concatenated.

Training and tuning happen in 02_nn_train.py and 03_nn_tuning.py.

D. Key Design Decisions

Decision	What I chose	Why
Evaluation metric	Custom Social Cost loop	GridSearchCV uses accuracy, which ignores the 5:1 cost asymmetry
Class imbalance	XGBoost: sample_weight; MLP: 5× oversampling	MLP lacks native class_weight; SMOTE creates unrealistic samples on one-hot data
Overfitting control	Early stopping (XGB: 50 rounds; MLP: 20 epochs)	Cheaper than full k-fold CV at 60K samples
Reproducibility	random_state=42 everywhere	Ensures repeatable results

VI. CLASSIC MACHINE LEARNING MODEL: XGBOOST

A. How XGBoost Works

XGBoost works by training decision trees sequentially. The first tree makes predictions on all 47,520 training samples, and each subsequent tree focuses on correcting the errors made by the previous ones. This process continues for up to 1,000 rounds, or stops earlier if early stopping is triggered after 50 rounds without improvement.

For this three-class problem, the model minimises multi-class log-loss, which strongly penalises confident but incorrect predictions. For instance, if the model assigns a high probability to the wrong class (e.g. predicting "Functional" with 90% confidence when the pump actually needs repair), the penalty becomes very large. In contrast, lower-confidence mistakes are penalised less. This behaviour is desirable in this context, since misclassifying a faulty pump as functional is a costly error in practice.

To address class imbalance, I applied the sample_weight parameter so that Class 2 errors are weighted W times more heavily. Initially, without weighting, the model tended to ignore Class 2, which only represents about 7% of the data. After setting $W = 5$, I observed that the model paid significantly more attention to these minority cases, rather than simply optimising for the majority class to achieve a lower overall loss.

B. Hyperparameters and Sensitivity Analysis

The Social Cost function was used as the final evaluation criterion, but it does not decide how the model is trained. The sensitivity analysis served a different purpose: it tested how different Class 2 training weights changed the model's behaviour. In other words, the Social Cost function measures the outcome after prediction, while the Class 2 weight influences the learning process during training.

There is no guaranteed best training weight in advance. Even though the project defines a 5:1 social cost for missed repairs, using a 5× training weight does not automatically guarantee the lowest final Social Cost. Therefore, the sensitivity analysis was needed to check whether 5× was actually the best practical setting compared with lower and higher weights.

```
BASE_PARAMS = {
    'n_estimators': 1000,
    'learning_rate': 0.05,
    'max_depth': 6,
    'objective': 'multi:softprob',
    'num_class': 3,
    'eval_metric': 'mlogloss',
    'use_label_encoder': False,
    'random_state': 42,
    'n_jobs': -1,
    'verbosity': 0,
    'early_stopping_rounds': 50
}

WEIGHTS_TO_AUDIT = [1, 5, 10, 20]
```

Figure 20: XGBoost base hyperparameters and class-weight settings used in the sensitivity analysis.

I used the following base hyperparameters:

Parameter	Value	Why I chose this
n_estimators	1000	Allows enough trees; early stopping prevents using too many
learning_rate	0.05	Small steps for stable learning
max_depth	6	Deep enough for feature interactions, not so deep that it overfits
early_stopping_rounds	50	Stop if validation loss does not improve for 50 rounds

The main tuning variable was the Class 2 weight. I tested weights of 1, 5, 10, and 20:

Weight	Accuracy	Class 2 Recall	Social Cost
1×	0.7923	0.2584	3,302
5×	0.7507	0.6385	**2,752**
10×	0.6959	0.7949	3,111
20×	0.6247	0.8598	3,965

Without any weighting (1×), the model missed 74% of pumps needing repair; recall was 0.26. As I increased the weight, recall climbed (0.86 at 20×), but false alarms climbed faster. Weight = 5 produced the lowest Social Cost at 2,752. Beyond that point, each additional true positive comes with so many extra false positives that total cost rises.

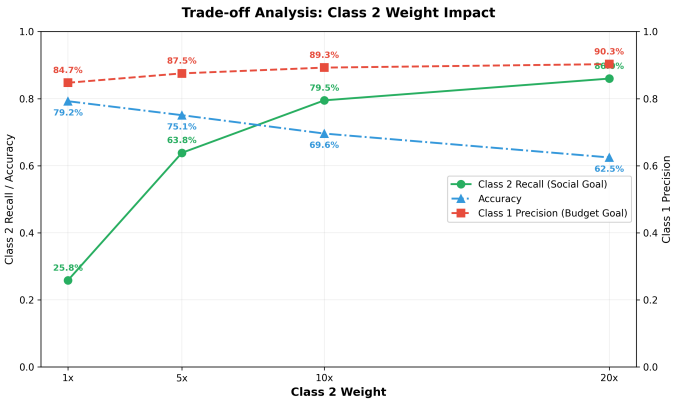


Figure 21: Trade-off between Class 2 weighting, recall, and accuracy.

C. Feature Importance

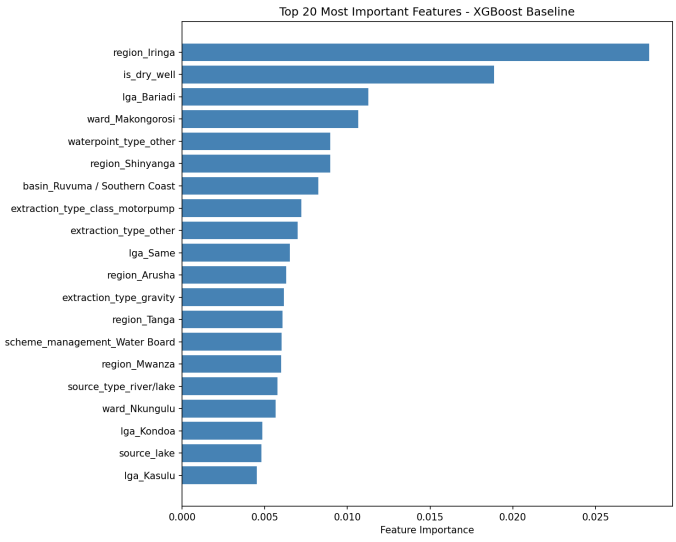


Figure 22: Top feature importances from the XGBoost model.

Geographic features dominate the importance ranking: region_Iringa is first, lga_Bariadi third. The most useful engineered feature is is_dry_well (rank 2, importance = 0.019), confirming that the "sure-kill" signal from the EDA carries predictive value. My interaction features (compound_risk_score, sure_kill_score) land around rank 64–

65 out of 2,400+ columns. They contribute, but XGBoost's own tree splits already capture most of those patterns.

VII. NEURAL NETWORK: MLP

A. Architecture and Training

```
HIDDEN_LAYER_SIZES = (128, 64, 32)

ACTIVATION = 'relu'
```

Figure 23: Three-layer funnel-shaped MLP

```
model = MLPClassifier(
    hidden_layer_sizes=HIDDEN_LAYER_SIZES,
    activation=ACTIVATION,
    solver=SOLVER,
    alpha=ALPHA,
    max_iter=MAX_ITER,
    early_stopping=EARLY_STOPPING,
    validation_fraction=VALIDATION_FRACTION,
    n_iter_no_change=N_ITER_NO_CHANGE,
    random_state=RANDOM_STATE,
    verbose=True
)
```

Figure 24: MLP Classifier uses these parameters

```
print(f"  Architecture:")
print(f"    Input Layer:  {X_train.shape[1]} neurons (auto-inferred)")
print(f"    Hidden Layer 1: {HIDDEN_LAYER_SIZES[0]} neurons (ReLU)")
print(f"    Hidden Layer 2: {HIDDEN_LAYER_SIZES[1]} neurons (ReLU)")
print(f"    Hidden Layer 3: {HIDDEN_LAYER_SIZES[2]} neurons (ReLU)")
print(f"    Output Layer: 3 neurons (Softmax implicit)")
```

Figure 25: MLP Architecture

```
ALPHA = 0.05

MAX_ITER = 500
EARLY_STOPPING = True
VALIDATION_FRACTION = 0.1
N_ITER_NO_CHANGE = 20
RANDOM_STATE = 42
```

```
print(f"    L2 Alpha: {ALPHA}")
print(f"    Early Stopping: {EARLY_STOPPING}")
```

Figure 26: L2 regularization and early stopping

```
print(f"    L2 Alpha: {ALPHA}")
print(f"    Early Stopping: {EARLY_STOPPING}")
print(f"    Max Iterations: {MAX_ITER}")

print("\n[STEP 3] Training MLP Model")
print("-" * 55)
print("  Training with early stopping...")
print("  (Will stop if no improvement for 20 iterations)\n")

train_start = time.time()
model.fit(X_train, y_train)
```

Figure 27: MLP training configuration for regularisation and validation.

```
y_pred = model.predict(X_val)
y_pred_proba = model.predict_proba(X_val)
```

Figure 28: MLP probability-output setup for the three-class task.

I chose a Multi-Layer Perceptron (MLP) with three hidden layers in a "funnel" shape:

Input: 2400+ features
|
Layer 1: 128 neurons (ReLU)
|
Layer 2: 64 neurons (ReLU)
|
Layer 3: 32 neurons (ReLU)
|
Output: 3 neurons (Softmax)

The input is large because one-hot encoding expands the pump metadata into more than 2,400 columns. I used a funnel shape because the first layer needs enough width to combine sparse categorical indicators, while the later layers force the model into a smaller representation before classification. ReLU was chosen because it is the default practical choice for hidden layers and trains faster than sigmoid or tanh on this kind of numeric matrix. In sklearn.MLPClassifier, the three output probabilities are produced by the built-in multiclass probability calculation; I did not implement a custom softmax layer myself. Since sklearn.MLPClassifier does not support class_weight, I handled the class imbalance by oversampling Class 2 five times before training. This is simple and matches the 5:1 social-cost idea, but it also creates duplicate rows, so I used L2 regularisation and early stopping to reduce memorisation.

B. Hyperparameter Tuning (*03_nn_tuning.py*)

I wrote a custom tuning loop (not GridSearchCV) so I could optimise for Social Cost instead of accuracy. I tested 10 combinations:

Parameter	Values tested	Why
alpha (L2 regularisation)	{0.0001, 0.001, 0.01, 0.05, 0.1}	Prevent memorisation of oversampled copies
learning_rate_init	{0.001, 0.0005}	Balance speed vs. stability
hidden_layer_sizes	(128, 64, 32) fixed	Keep architecture constant
early_stopping	True, patience = 15	Stop when validation loss plateaus

I first fixed the architecture at 128-64-32 and varied the L2 regularisation strength. I tested five alpha values and two learning rates. For each combination, I trained the model and calculated the Social Cost on the validation set.. I did not tune hidden-layer size, activation,or oversampling factor;that keeps the search small enough to explain, but it also meansthe MLP design is not fully optimised.

Top results (sorted by Social Cost):

Alpha	Learning Rate	Social Cost	Accuracy
0.05	0.001	**2,879**	0.7501
0.1	0.001	2,880	0.7499
0.1	0.0005	2,882	0.7554

The best configuration was $\alpha = 0.05$ with learning rate 0.001, giving a Social Cost of 2,879. Moderate-to-strong regularisation consistently beat the weaker settings. The model trains on five

copies of every Class 2 sample, so without enough L2 penalty it memorises the duplicates rather than learning the underlying pattern.

VIII. EXPERIMENTS, RESULTS & COMPARISON

A. TR Compliance Audit

I evaluated each XGBoost weight configuration against all five technical requirements (model_audit.py):

Weight	Social Cost	FN	FP	Verification	TR Status
1×	3,302	640	102	$5 \times 640 + 102 = 3,302$	FAIL (TR-01)
5×	2,752	312	1,192	$5 \times 312 + 1,192 = 2,752$	ALL PASS
10×	3,111	177	2,226	$5 \times 177 + 2,226 = 3,111$	ALL PASS
20×	3,965	121	3,360	$5 \times 121 + 3,360 = 3,965$	ALL PASS

The unweighted model (1×) fails TR-01 because its Class 2 recall is only 0.26 (below the 0.50 threshold). Weights 5×, 10×, and 20× all pass, but 5× has the lowest Social Cost. Higher weights keep reducing false negatives but generate so many false positives that the total cost rises.

B. Confusion Matrix Analysis

XGBoost (Weight 5×)

	Pred: Functional	Pred: Non-Functional	Pred: Needs Repair
Actual: Functional	5,228	383	841
Actual: Non-Functional	1,075	3,139	351
Actual: Needs Repair	248	64	551

The most common misclassification is XGBoost labelling a "Needs Repair" pump as "Functional" (248 cases). A degrading pump may still produce water, so its metadata can look almost identical to a healthy one. For maintenance teams this is the costliest error, because the pump keeps running until it breaks completely. The model catches 551 out of 863 Class 2 pumps (recall = 0.64), up from 0.26 without any weighting.

MLP ($\alpha = 0.05$)

	Pred: Functional	Pred: Non-Functional	Pred: Needs Repair
Actual: Functional	5,072	569	811
Actual: Non-Functional	930	3,318	317
Actual: Needs Repair	248	102	513

The MLP shows a similar error pattern, but it catches fewer Class 2 pumps than XGBoost: 507 versus 551. This means it misses more pumps that are already in the "Needs Repair" category, which is undesirable for a maintenance-prioritisation system.

C. Main Results

Metric	XGBoost (W=5×)	MLP ($\alpha = 0.05$)
Social Cost	**2,752**	2,879
FN (Class 2)	312	350
FP (Class 2)	1,192	1,129
Accuracy	0.7507	0.7501
Class 2 Recall	0.6385	0.5944
Class 1 Precision	0.8753	0.8351
Class 2 Lift	8.78×	8.18×
Overfit Gap	0.038	0.122
TR-01 (Recall > 0.50)	✓	✓
TR-02 (Precision > 0.80)	✓	✓
TR-03 (Overfit < 0.10)	✓	× (0.122)
TR-04 (Confidence)	✓	✓
TR-05 (Lift > 3×)	✓	✓

D. Overfitting Analysis

The MLP shows an overfitting gap of 0.122, which technically fails TR-03 (<0.10). This gap is likely an artifact of the oversampling rather than real memorisation. The training set contains five copies of every Class 2 sample, so training accuracy is measured on an artificially balanced distribution. The validation set uses the original proportions.

To check for real overfitting, I looked at the learning curves:

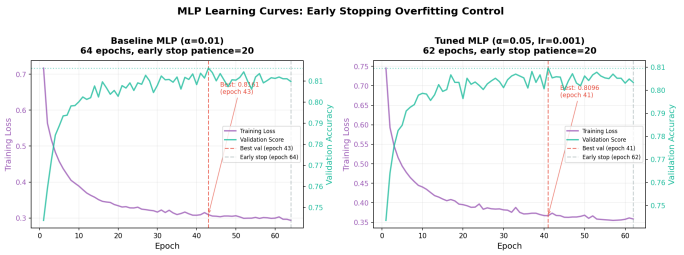


Figure 29: Learning curves for the baseline and tuned MLP models.

Both the baseline and tuned MLP show validation loss flattening out without diverging from training loss, ruling out classic overfitting. Early stopping (patience = 20) halts training at the plateau. The gap is a distribution artefact, not a capacity problem.

In practice, this means I would not deploy the MLP without changing the evaluation setup or retraining strategy.If I had more time, I would have tried using a cost-based overfitting metric (comparing Social Cost on train and validation) instead of accuracy, which would better reflect the metric I actually care about.

E. Probabilistic Confidence Output (TR-04)

Both models produce a probability distribution $P(y = c \mid \mathbf{x})$ over all three classes via softmax. Rather than a flat label, the output is a confidence score per class (e.g., 91% chance of needing repair). Maintenance teams can rank pumps by predicted urgency and dispatch crews to the highest-probability cases first.

Sample predictions (XGBoost W=5×):

Sample	P(Func)	P(Non-F)	P(Repair)	Predicted	Actual	Correct
#8333	0.006	0.990	0.004	Non-Func	Non-Func	✓
#11848	0.047	0.042	0.911	Repair	Repair	✓
#4577	0.211	0.067	0.722	Repair	Functional	×
#6735	0.333	0.245	0.421	Repair	Non-Func	×

The confidence scores show how the model output can support maintenance prioritisation rather than automatic decision-making. Very high-confidence cases are reliable in these examples: sample #8333 is correctly predicted as Non-Functional with 0.990 confidence, and #11848 is correctly predicted as Needs Repair with 0.911 confidence. However, the wrong predictions show why a human-in-the-loop workflow is still necessary. Sample #6735 is a low-confidence prediction, with only 0.421 probability for Repair, so it should clearly be flagged for review. Sample #4577 is more problematic: the model predicts Repair with 0.722 confidence, but the pump is actually Functional. This shows that confidence is useful but not sufficient on its own.

In deployment, I would therefore use probability scores as a triage mechanism. Very high-confidence repair predictions could enter the inspection queue directly, while medium- and low-confidence cases should be reviewed by a local officer, especially when they are outside hotspot regions or when the predicted class conflicts with other operational knowledge.

F. Strengths and Limitations

XGBoost

Strengths: XGBoost worked well with the mixed feature types in the dataset, including numerical, categorical, and boolean variables. It also did not require feature scaling, which made the preprocessing simpler than for the MLP model. Another advantage was the sample_weight parameter. This allowed me to make Class 2 errors more costly during training without changing the original data distribution. The built-in feature importance was also useful because it highlighted regions and water sources that may deserve more attention from field teams.

Limitations: One limitation is that there is no obvious best value for the Class 2 weight. I had to test several values and compare the results. The feature importance rankings should also be interpreted carefully, because gain-based importance can favour high-cardinality features such as region, especially after one-hot encoding. A better follow-up would be to use permutation importance for a fairer comparison. XGBoost is also weak at extrapolating beyond the training data. For example, if a pump is much older than the pumps seen during training, the model may not make a reliable prediction.

MLP

Strengths: The MLP can learn feature interactions that I did not manually design. This is useful because the condition of a pump may depend on combinations of features rather than on single variables alone. The predicted probabilities were also useful for ranking pumps by urgency, even though I did not manually implement a softmax layer.

Limitations: The MLP required more preprocessing than XGBoost. Scaling, encoding, cardinality filtering, and oversampling all had to be done before training. It also took around five times longer to train. Another issue is that oversampling Class 2 creates duplicate rows, which can make the training accuracy look better than it really is. This produced a larger overfitting gap. I would have preferred to use a native class_weight parameter, but sklearn.MLPClassifier does not support it.

G. What I Would Improve

Three changes would improve the pipeline:

1. Redesign TR-03 to check generalisation using both accuracy and Social Cost. The MLP currently fails the accuracy-based TR-03, and this should not be ignored because it indicates a deployment risk. However, this case study is mainly driven by Social Cost rather than raw accuracy, so future evaluation should also compare train and validation Social Cost on the original non-oversampled data. This would not change the current result, but it would make the overfitting check better aligned with the maintenance objective.

2. For future improvement, I should not rely only on XGBoost's built-in gain importance. Features such as region have many categories, so XGBoost may give them a high importance score simply because they offer many possible splits. To avoid over-interpreting this, I should use permutation importance or SHAP values to check whether these features really improve the model's predictions.

3. Run 5-fold cross-validation for XGBoost instead of relying on a single 80/20 split. A different random seed could shift the results, and cross-validation would give more stable estimates.

IX. CONCLUSION

XGBoost with weight = 5 is my final recommendation. It has the lowest Social Cost (2,752) and is the only option that both minimises the cost objective and passes all five technical requirements. The MLP comes close in overall accuracy and Social Cost, but it still costs 127 points more and fails TR-03 because the oversampled training distribution makes the accuracy gap exceed the threshold.

A likely reason for this difference is how the two models handle class imbalance. XGBoost uses sample_weight, so the loss function becomes cost-sensitive while the training distribution itself stays unchanged. The MLP uses physical oversampling, which copies Class 2 samples five times. This helps recall, but it also makes the training accuracy less comparable to validation accuracy. For this case study, that makes XGBoost the safer deployment choice: it gives the better cost trade-off without changing the structure of the dataset.

A. Transferability

This pipeline could transfer best to maintenance problems that look similar to the water pump dataset: static tabular records, imbalanced failure classes, and a clear cost difference between missed failures and unnecessary inspections. For that reason, I would not treat the exact XGBoost result as a general rule. What transfers is mainly the evaluation logic: define the cost of each mistake first, then tune the model around that cost instead of only optimising accuracy.

A closer example would be agricultural irrigation systems. The available data would probably be similar to this project, such as location, equipment type, installation year, water source, maintenance history, and current condition. In that case, the same cleaning, feature engineering, class weighting, and Social Cost evaluation could be reused with only small changes to the feature names and class labels.

I would be more careful about applying the same pipeline to sensor-based maintenance, such as electrical grid monitoring or machines with vibration and pressure readings. Those problems depend on changes over time, while my model uses mostly static pump metadata. In that setting, XGBoost might still be a useful baseline, but models designed for time-series patterns would probably be more appropriate.

REFERENCES

1. Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? *NeurIPS*, 35. arxiv.org/abs/2207.08815

2. Shwartz-Ziv, R., & Armon, A. (2022). Tabular data: Deep learning is not all you need. *Information Fusion*, 81, 84–90. arxiv.org/abs/2106.03253

3. Chawla, N. V. et al. (2002). SMOTE: synthetic minority over-sampling technique. *JAIR*, 16, 321–357.

4. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. deeplearningbook.org

5. Danert, K. (2009). *Cost-effective boreholes in sub-Saharan Africa*. RWSN & UNESCO.

6. DrivenData. (2015). Pump it Up: Data Mining the Water Table. drivendata.org

APPENDIX A: REPRODUCTION INSTRUCTIONS

A.1 Operating Environment

Component	Specification
OS	Windows 10/11
Python	3.10+
Key libraries	pandas, numpy, scikit-learn, xgboost, matplotlib, seaborn, pyarrow

All experiments use `random_state=42` for reproducibility.

A.2 Core Scripts

The pipeline uses 8 core scripts, executed in order:

```
# Step 1: Clean raw data
python src/data_cleaning.py
# Input: training_set_values.csv, training_set_labels.csv, test_set_values.csv
# Output: cleaned_train_values.csv, cleaned_train_labels.csv, cleaned_test_values.csv

# Step 2: Exploratory analysis for risk signals
python src/strategic_eda.py
# Input: cleaned_train_values.csv, cleaned_train_labels.csv
# Output: EDA evidence and plots used to motivate risk-based features

# Step 3: Feature engineering
python src/feature_engineering.py
# Input: cleaned CSV files
# Output: final_X_train.parquet, final_y_train.csv, final_X_test.parquet

# Step 4: XGBoost training, class-weight sensitivity analysis, and social-cost audit
python src/model_audit.py
# Input: final_X_train.parquet, final_y_train.csv
# Output: audit_results.csv, audit_tradeoff_summary.png, confusion-matrix plots

# Step 5: MLP preprocessing
python src/01_nn_data_prep.py
# Input: final_X_train.parquet, final_y_train.csv
# Output: nn_preprocessed_data.pkl

# Step 6: MLP hyperparameter tuning
python src/03_nn_tuning.py
# Input: nn_preprocessed_data.pkl
# Output: nn_tuning_results.csv

# Step 7: Final MLP training and validation audit
python src/02_nn_train.py
# Input: nn_preprocessed_data.pkl
# Output: nn_model.pkl, nn_audit_results.csv, nn_confusion_matrix.png,
        nn_training_curve.png

# Step 8: Probabilistic confidence verification
python src/06_confidence_analysis.py
# Input: final_X_train.parquet, final_y_train.csv, nn_model.pkl
# Output: confidence_samples.csv, confidence_distribution.png
```

Total estimated runtime: ~15 minutes on consumer hardware.

A.3 Data Acquisition

The dataset is available from the DrivenData competition "Pump It Up: Data Mining the Water Table" (https://www.drivendata.org/competitions/7/pump-it-up-data-mining-the-water-table/). Download `training_set_values.csv`, `training_set_labels.csv`, and `test_set_values.csv`.